

Introduction to Social Data Analytics: Lecture 3 Part 1

Police Stops

- ▶ The most common way civilians interact with police are through traffic stops (there are about 50,000 a day)
- ▶ Detecting and eliminating racial discrimination in police behavior is an important policy goal, but can be difficult to detect without data
- ▶ **The Stanford Open Policing Project** is aggregating data on police stops across states and cities so that researchers and journalists and policymakers can improve interactions between police and the public
- ▶ The data so far has information on about **200 million stops**
 - ▶ Tons of potential applications for this data

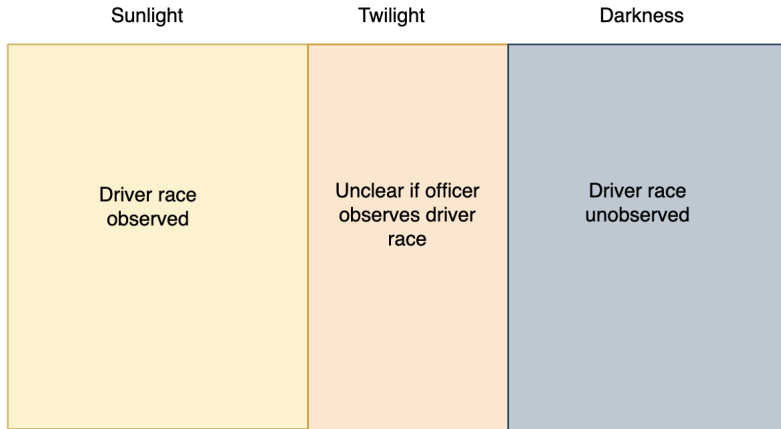
Racial Disparities vs. Racial Discrimination

- ▶ Racial disparities are any differences between groups of different races
- ▶ Many factors can cause racial disparities, but common focus is to try to understand if the disparity is driven by racial discrimination
 - ▶ In our example, we are interested in exploring racial discrimination on the part of police officers making traffic stops
- ▶ There are many methods that have been proposed to detect racial discrimination. Today we focus on one: **the veil of darkness test**

Veil of Darkness test for Discrimination

- ▶ Veil of darkness intuition: it is more difficult to observe race in darkness
 - ▶ Therefore, if officers are racially profiling, traffic stops for minorities should decrease in darkness
- ▶ We are going to implement this test in San Diego by combining data from the **Stanford Open Policing Project (SOPP)** to data on sunset times

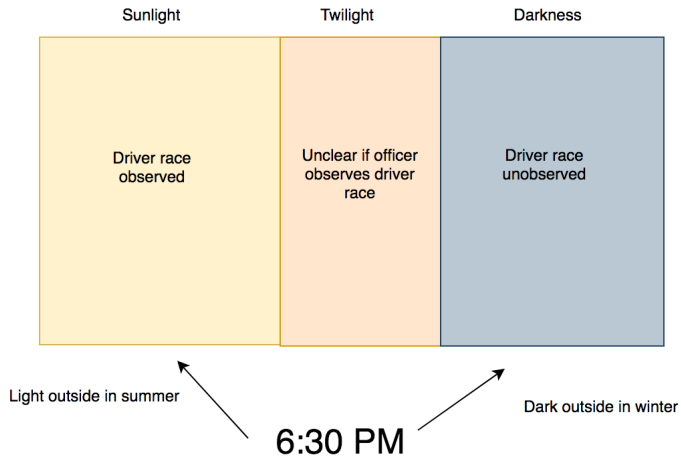
Graphical Illustration



The veil of darkness test

- ▶ Comparing stop rates by race at different times might be misleading
 - ▶ Drivers on the road at 12 noon (sunlight) may be very different than drivers at 12 midnight (darkness)
- ▶ Grogger and Ridgeway (2006) focus on the **intertwilight period**, which are times in which it is sometimes light out and sometimes dark

The intertwilight period



The veil of darkness test

- ▶ Grogger and Ridgeway (2006) use this study to study differences in stop rates for Black vs. non-Black drivers
- ▶ To understand how the test works, let's consider a single time: 6:30 PM

The veil of darkness test

- ▶ Grogger and Ridgeway (2006) use this study to study differences in stop rates for Black vs. non-Black drivers
- ▶ To understand how the test works, let's consider a single time: 6:30 PM
- ▶ Imagine **40 percent** of drivers stopped are Black when it is light out (at 6:30 PM)
- ▶ But **35 percent** of drivers stopped are Black when it is dark out (at 6:30 PM)
- ▶ In this hypothetical example, the decrease in stops for Black drivers must be due to officers inability to racial profile when it is dark

Data requirements

- ▶ From the SOPP data we have information on the time the traffic stop occurs (but the data comes in 2 separate datasets)
- ▶ We also have another dataset on sunset times
- ▶ Therefore we need to combine these datasets to perform our analysis
- ▶ This will introduce us to **data wrangling** in Stata

What is data wrangling?

- ▶ When you start an empirical research project in the social sciences, you must ask the question: what data can I use to answer this question?
- ▶ Once you have identified the data, it rarely comes in a format that is ready for data analysis
- ▶ Generally, there are a number of steps that must be taken to prepare the data before analysis
- ▶ These steps are often referred to as **data wrangling**

Examples of data wrangling (1/2)

- ▶ The Census has confidential income data that is available to researchers that apply
- ▶ The data, however, is stored separately for every state
- ▶ To create a master dataset, you would need to combine all these individual datasets together
- ▶ In Stata, this is accomplished with the **append** command

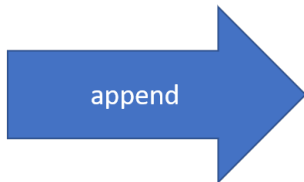
Examples of data wrangling (2/2)

- ▶ Imagine you are interested in the relationship between economic conditions and the opioid crisis in the United States
- ▶ The Bureau of Labor Statistics provides county-level information on unemployment
- ▶ The Centers for Disease Control and Prevention (CDC) provides county-level opioid prescription data
- ▶ To study this question, we would need to combine these two datasets together at the county level
- ▶ In Stata, this is accomplished with the **merge** command

Illustrative example (Append Command)

student	school	gpa
1	A	3.3
2	A	3.2

student	school	gpa
3	B	2.9
4	B	3.0



student	school	gpa
1	A	3.3
2	A	3.2
3	B	2.9
4	B	3.0

Append Command

- ▶ If class A data is stored in `classA.dta` and class B data is stored in `classB.dta` we can combine them by typing

```
use classA.dta, clear
append using classB.dta
```

- ▶ Note: both `classA.dta` and `classB.dta` need to be in the working directory for this command to work
- ▶ You can continue to append more datasets if there are additional classrooms

Merge Command

- ▶ The merge command is used when you want to **add columns** to a dataset
- ▶ We are often interested in relationships between different variables
- ▶ But if the variables are not in the same dataset, we need a way to combine datasets
 - ▶ A lot of cutting-edge research in social sciences stems from merging together previously unlinked datasets to observe new relationships
- ▶ There are two types of merges in Stata
 - ▶ The type of merge depends on the number of unique observations per the matching variable in each dataset

Example of a 1:1 merge

student	school
1	A
2	A
3	B
4	B

student	gpa
1	3.3
2	3.2
3	2.9
4	3.0

merge 1:1 student

student	school	gpa
1	A	3.3
2	A	3.2
3	B	2.9
4	B	3.0

- ▶ In each dataset, there is a unique value for `student` for every observation
- ▶ This is therefore a one-to-one merge (we are matching each student in the `school` dataset to one student in the `gpa` dataset)

Example of a m:1 merge

student	school	gpa
1	A	3.3
2	A	3.2
3	B	2.9
4	B	3.0

school	pop
A	1411
B	2692

merge m:1 school

student	school	gpa	pop
1	A	3.3	1411
2	A	3.2	1411
3	B	2.9	2692
4	B	3.0	2692

- ▶ In the first dataset, there are multiple observations per school (where school is the variable we are merging on)
- ▶ This is therefore a many-to-one merge (we are matching many schools observations in the school dataset to one school in the population dataset)

Merging in Stata

- ▶ There are two steps to merging data in Stata
 - ▶ Step 1: load one of the two datasets. Whichever dataset you load now is referred to as the **master dataset**
 - ▶ Step 2: Use the `merge` command to combine the master dataset with the second of the two datasets. The dataset you are merging to is called the **using dataset**

Merging in Stata (basic syntax)

- Let's first try a 1:1 merge where we have the two datasets (school.dta and gpa.dta)

	student	school		student	gpa
1	1	A	1	1	3.3
2	2	A	2	2	3.2
3	3	B	3	3	2.9
4	4	B	4	4	3

Merging in Stata (basic syntax)

```
/*set directory to where data is stored*/  
cd /Users/davidarnold/Dropbox/Teaching/EP5/online/03_week/data  
  
/*whichever dataset is loaded here becomes the master dataset*/  
use school.dta, clear  
  
/*merge to the using dataset*/  
merge 1:1 student using gpa.dta
```

```
/Users/davidarnold/Dropbox/Teaching/EP5/online/03_week/data
```

Result	Number of obs
Not matched	0
Matched	4 (_merge==3)

Merging in Stata (m:1)

- Now let's first try a m:1 (many-to-one) merge where we have the two datasets (school.dta and pop.dta)

	student	school
1	1	A
2	2	A
3	3	B
4	4	B

	school	pop
1	A	1411
2	B	2692

Merging in Stata (m:1 merge)

```
/*set directory to where data is stored*/  
cd /Users/davidarnold/Dropbox/Teaching/EP5/online/03_week/data  
  
/*whichever dataset is loaded here becomes the master dataset*/  
use school.dta, clear  
  
/*merge to the using dataset*/  
merge m:1 school using pop.dta
```

```
/Users/davidarnold/Dropbox/Teaching/EP5/online/03_week/data
```

Result	Number of obs
Not matched	0
Matched	4 (_merge==3)

Merging (one-to-many)

- ▶ A one-to-many merge (1:m) is exactly the same as a many-to-one (m:1)
- ▶ In our example, if we had loaded `pop.dta` first, then we would have specified `1:m` instead of `m:1`
 - ▶ This is because there is a single observation per school in `pop.dta` and multiple the `school.dta`
- ▶ The syntax for this would be

```
use pop.dta, clear
merge 1:m school using school.dta
```


Imperfect Merges

- Sometimes two datasets will not merge perfectly

	student	test_score		student	hw_score
1	1	82	1	2	99
2	2	74	2	3	82
3	3	93	3	4	83
4	4	91	4	5	90
5	5	91	5	6	70

- In this example, student 1 has a test score, but no homework score, while student 6 has a homework score but no test score
- Let's see what happens if we merge these two datasets in Stata

```

/*load test score data*/
use test_score.dta, clear

/*merge to the hw data*/
merge 1:1 student using hw_score.dta

```

Result	Number of obs	

Not matched	2	
from master	1	(_merge==1)
from using	1	(_merge==2)
Matched	4	(_merge==3)

Result	Number of obs		

Not matched	2		
from master	1	(<code>_merge==1</code>)	
from using	1	(<code>_merge==2</code>)	
Matched	4	(<code>_merge==3</code>)	

- ▶ 2 observations were not matched
- ▶ 1 from the master dataset (`test_score.dta`) and the other from the using dataset (`hw_score.dta`)
- ▶ A new variable (named `_merge`) has been added to the dataset to keep track of matched vs. unmatched observations. It is equal to 1 for observations only in the master dataset, 2 for observations only in the using, and 3 for observations in both

Keeping only matched variables

- ▶ Whether you keep unmatched observations in your dataset depends a bit on your analysis
- ▶ If you really need both homework score and test score for all of your analysis, you can probably safely keep only matched variables
- ▶ To do this in Stata after a merge command, we can type

```
keep if _merge==3
```

(2 observations deleted)

The Veil of Darkness Test

- ▶ To apply the veil of darkness test we are going to look at the time period 6:30-7:00 PM.
- ▶ Sometimes it is dark out at 6:30-7:00 PM, sometimes it is light. We need to identify whether it was light or dark for every stop in our data
- ▶ Problems:
 - ▶ Our data for stops currently comes in two separate files, one for the years 2014-2015 and one for the years 2016-2017
 - ▶ Our sunset data comes in a separate file

Creating a Dataset of All Stops

- ▶ To create a dataset of all stops between 2014-2017, we need to append our data together

/Users/davidarnold/Dropbox/Teaching/EP5/online/03_week/data

```
use san_diego_stops_2014_2015.dta, clear  
append using san_diego_stops_2016_2017.dta
```

Merging to sunset data

- ▶ Next, we need to merge our data to sunset times
- ▶ To do this, we need to make sure of three things
 - ▶ Is there a variable that we can merge on?
 - ▶ Is this a one-to-one or many-to-one merge?
 - ▶ Is the merging variable in both datasets stored in the same format?

One-to-one vs. many-to-one

- ▶ We are merging on a variable called date
- ▶ Our traffic data is at the stop level. There are multiple stops a day, so we know there will not be a unique date for every observation
- ▶ We still need to check that there is a unique date for every observation in the sunset data (which seems likely, but you should always check this)

```
use sd_sunset.dta, clear  
unique date
```

Number of unique values of date is 1186

Number of records is 1186

- ▶ The number of unique values is equal to the number of records (or observations), so this will be a many-to-one merge

Unique Command

- ▶ Many of you may have received an error code “unique not found”
- ▶ `unique` is actually a command written by a user and made available to others
- ▶ To install the `unique` command type

```
ssc install unique
```

- ▶ In general, you might find a command online that will help you perform some data analysis. Many commands can be installed by typing `ssc install commandname`

Checking formats

- ▶ Also need to check the format of the dates is the same in both datasets
- ▶ In our case, the dates are held in the same format, so we are now ready to merge the datasets together

Merging the data

- Now let's merge our stops data (currently in memory) to the sunset data

```
/* merge but only keep dates in both datasets */  
merge m:1 date using sd_sunset.dta, keep(3) nogen
```

```
/* save dataset named sd_analysis_sample in the working directory */  
save sd_analysis_sample.dta, replace
```

Result	Number of obs
Not matched	0
Matched	381,149

file sd_analysis_sample.dta saved